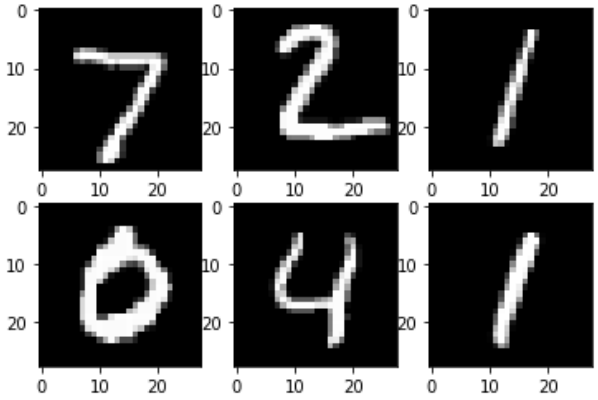
Μηχανική Μάθηση

Εργασία 2: Νευρωνικά Δίκτυα

Η εργασία αποσκοπεί στην χρήση νευρωνικών δικτύων για την αυτόματη αναγνώριση χειρόγραφων ψηφίων. Θα χρησιμοποιηθεί η βάση MNIST[1] που περιλαμβάνει grayscale εικόνες χειρόγραφων ψηφίων σε διάσταση 28x28 pixels. Το dataset είναι χωρισμένο σε training set (60.000 εικόνες) και test set (10.000 εικόνες). Μερικά παραδείγματα εικόνων φαίνονται παρακάτω:



Επίσης δίνονται 3 νευρωνικά δίκτυα, υλοποιημένα σε PyTorch[2], ένα από τα πιο δημοφιλή και ισχυρά πακέτα λογισμικού σχετικά με deep learning. Πιο συγκεκριμένα, έχουν υλοποιηθεί:

1. Ένα Multi-Layer Perceptron (**MLP**) με τα εξής επίπεδα:
   i. Κρυμμένο επίπεδο με διάσταση=100.
   ii. Softmax
2. Ένα Convolutional Neural Network (**CNN**) που αποτελείται από τα εξής επίπεδα:
   i. Convolutional layer: 32 κανάλια (φίλτρα), kernel=3x3, stride=1.
   ii. Max-pooling: kernel=2x2.
   iii. ReLU
   iv. Convolutional layer: 64 κανάλια, kernel=3x3, stride=1.
   v. Max-pooling: kernel=2x2.
   vi. ReLU
   vii. Dense (linear) layer: διάσταση εισόδου=1600, διάσταση εξόδου=10.
   viii. Softmax
3. Ένα προ-εκπαιδευμένο δίκτυο (**MobileNetV2**) που έχει εκπαιδευτεί στο τεράστιο σύνολο εικόνων ImageNet[3]. Έχει περίπου 3.5 εκατομμύρια παραμέτρους και βασίζεται στην αρχιτεκτονική CNN ενώ είναι σχεδιασμένο για χρήση σε κινητές και ενσωματωμένες συσκευές με περιορισμένους υπολογιστικούς πόρους.

Η καθεμία υλοποίηση αφού διαβάζει τα δεδομένα του dataset, χρησιμοποιεί το training set για την εκπαίδευση του δικτύου και ελέγχει την αποτελεσματικότητα του τελικού δικτύου στο test set. Οι ακόλουθες υπερ-παράμετροι χρησιμοποιούνται και στα 3 δίκτυα:

- Mini-batch size = 100
- Learning rate = 0.001

[1] https://en.wikipedia.org/wiki/MNIST_database
[2] https://pytorch.org/get-started/locally/
[3] https://www.image-net.org/

- Epochs = 5 για τα MLP και CNN, 3 για το MobileNetV2
- Optimizer: Adam
- Loss function: Cross-entropy (negative log likelihood)

Ερωτήματα

1. Αφού εξοικειωθείτε με τον κώδικα, εκτελέστε τα καθένα από τα 3 προγράμματα με χρήση GPU και καταγράψτε τα αποτελέσματα του classification accuracy στο test set. Επαναλάβετε την εκτέλεση 3 φορές και καταγράψτε τον μέσο όρο του accuracy και την τυπική απόκλιση (standard deviation) στο test set. Επίσης, καταγράψτε τον απαιτούμενο χρόνο για την εκπαίδευση του κάθε μοντέλου και ξεχωριστά τον χρόνο που απαιτείται για την εφαρμογή του εκπαιδευμένου μοντέλου στο test set.
2. Εκτελέστε τον κώδικα μία μόνο φορά για το κάθε δίκτυο χωρίς χρήση GPU και συγκρίνετε τους χρόνους εκπαίδευσης και εφαρμογής με αυτά του ερωτήματος 1.
3. Για καθένα από τα 3 δίκτυα, επιλέξτε μία συγκεκριμένη εκτέλεση και βρείτε σε ποιες εικόνες από το test set κάνουν λάθος πρόβλεψη τα δίκτυα. Στην αναγνώριση ποιων ψηφίων αποτυγχάνει συνήθως το κάθε μοντέλο; Εμφανίστε μερικές τέτοιες εικόνες ψηφίων που έχουν αναγνωριστεί λάθος από το κάθε μοντέλο. Πόσες εικόνες ταξινομούνται λάθος και από τα 3 δίκτυα;
4. Δημιουργήστε ένα νέο MLP με δύο κρυμμένα επίπεδα και συνάρτηση ενεργοποίησης ReLU στο καθένα. Η διάσταση του πρώτου κρυμμένου επιπέδου να είναι 500 και του δεύτερου 200. Επίσης να εφαρμόζεται dropout με πιθανότητα 0.2 μετά από κάθε κρυμμένο επίπεδο (μετά το ReLU). Πώς συγκρίνονται τα αποτελέσματα με αυτά του ερωτήματος 1;
5. Στο αρχικό MLP προσθέστε dropout μεταξύ του κρυμμένου επιπέδου (μετά την ενεργοποίηση ReLU) και της εξόδου. Χρησιμοποιείστε το 10% του training set ως validation set για να βρείτε τον καλύτερο συνδυασμό των 2 υπε-παραμέτρων: πιθανότητα του dropout (μεταξύ των 0.1, 0.2, 0.3, 0.4, 0.5) και διάσταση του hidden layer (μεταξύ των 200, 400, 600, 800, 1000). Με βάση τον συνδυασμό που θα προκύψει, εκπαιδεύστε το δίκτυο με την ένωση του training και του validation set και βρείτε τον μέσο όρο της επίδοσης στο test set μετά από 3 επαναλήψεις. Πώς συγκρίνονται τα αποτελέσματα με αυτά του ερωτήματος 1;
6. Ο κώδικας του MobileNetV2 που δόθηκε δεν τροποποιεί τα βάρη του προ-εκπαιδευμένου δικτύου (frozen weights) κατά την διάρκεια της εκπαίδευσής του στην εργασία αναγνώρισης εικόνων ψηφίων. Τροποποιήστε τον κώδικα ώστε ολόκληρο το προ-εκπαιδευμένο δίκτυο να συνεχίσει την εκπαίδευσή του (fine-tuning) στο training set. Επαναλάβετε τη διαδικασία 3 φορές και συγκρίνετε τον μέσο όρο της επίδοσης στο test set με αυτή του ερωτήματος 1.
7. Τροποποιήστε τον αρχικό κώδικα του MobileNetV2 ώστε μόνο τα 3 τελευταία επίπεδα του προ-εκπαιδευμένου δικτύου να συνεχίσουν την εκπαίδευσή τους (fine-tuning) στο training set. Επαναλάβετε τη διαδικασία 3 φορές και συγκρίνετε τον μέσο όρο της επίδοσης στο test set με αυτή του ερωτήματος 1.
8. Δημιουργήστε ένα πίνακα με τα συγκεντρωτικά αποτελέσματα όλων των προηγούμενων πειραμάτων. Σχολιάστε τα αποτελέσματα που προέκυψαν. Ποιο μοντέλο δικτύου και με ποιες προδιαγραφές είναι η καλύτερη επιλογή; Είναι στατιστικά σημαντική η διαφορά της επίδοσης του καλύτερου μοντέλου από το δεύτερο καλύτερο; Ποιος τύπος δικτύου είναι πιο γρήγορος στην εκπαίδευσή του και ποιος είναι πιο γρήγορος στην εφαρμογή του στα δεδομένα του test set; Πόσο χρήσιμη είναι η GPU για την εκπαίδευση των 3 τύπων δικτύου;

Οδηγίες:

- Για να εξοικειωθείτε πιο εύκολα με τον κώδικα διαβάστε πρώτα μία σύντομη και περιεκτική εισαγωγή στη δομή των προγραμμάτων του PyTorch[4].
- Για την εκτέλεση του κώδικα συστήνεται να χρησιμοποιήσετε μία online πλατφόρμα όπως το Google Colab[5] ή το Kaggle[6] οι οποίες δίνουν την δυνατότητα και για χρήση GPU δωρεάν (για περιορισμένο διάστημα).
- Τα μοντέλα MLP και CNN μόνο στο 1ο ερώτημα απαιτείται να εκτελεστούν με χρήση GPU. Το MobileNetV2 μόνο στο 2ο ερώτημα απαιτείται να εκτελεστεί χωρίς GPU (μπορεί να διαρκέσει μερικές ώρες).
- Την πρώτη φορά που θα εκτελέσετε τον κώδικα θα γίνει download το MNIST dataset.
- Για τον διαχωρισμό του 10% του αρχικού training set σε validation set συστήνεται η χρήση της συνάρτησης **random_split** που περιλαμβάνεται στο **torch.utils.data**

Τι πρέπει να παραδώσετε:

Μία αναφορά (PDF) όπου να φαίνονται οι απαντήσεις, πίνακας αποτελεσμάτων, κλπ. που αφορούν στις ερωτήσεις της εργασίας. Επίσης, να επισημαίνεται ποια κομμάτια του αρχικού κώδικα τροποποιήθηκαν σε κάθε ερώτηση.

[4] https://pytorch.org/tutorials/beginner/basics/intro.html
[5] https://research.google.com/colaboratory/faq.html
[6] https://www.kaggle.com/code